

A project report on

CaRAG: Categorical Routing Augmented Generation Platform

by

Anuj Sharma

Internship Details

Mentor – Ms. Manisha Singh

Start date – 20th May 2026

End date – 15 July 2026

ABSTRACT

The rapid adoption of Large Language Models (LLMs) has created a new challenge for enterprises: how to leverage generative AI while maintaining factual accuracy, security, and strict data isolation. Although Retrieval-Augmented Generation (RAG) has emerged as the dominant approach for grounding LLM responses in enterprise knowledge, conventional RAG systems often struggle with semantic noise, poor exact-keyword retrieval, cross-tenant security concerns, and hallucinated responses when retrieval quality is insufficient.

This report presents the design and implementation of **CaRAG (Categorical Routing Augmented Generation)**, a category-aware, multi-tenant retrieval platform developed during an internship at **Jio Platforms Limited (Jio Cortex AI)**. The platform was engineered from first principles to address the limitations of traditional RAG systems by introducing intelligent search-space reduction, deterministic retrieval boundaries, and confidence-driven answer generation.

The final architecture evolved into a layered retrieval engine consisting of category-aware routing, hybrid retrieval using Dense Embeddings and BM25, Reciprocal Rank Fusion (RRF), Cross-Encoder reranking, HNSW-based vector indexing, and a Retrieval Confidence Gate designed to prevent low-confidence responses from reaching the language model. In parallel, a dedicated multi-tenant adapter layer was developed to provide JWT-based authentication, group-scoped document isolation, and real-time observability through WebSocket communication.

Beyond implementation, the project involved extensive system design decisions, architectural refactoring, debugging of vector database infrastructure, asynchronous processing optimization, multi-tenant security enforcement, and retrieval quality research. The resulting platform demonstrates how enterprise-grade AI systems are fundamentally retrieval and systems-engineering problems rather than solely language-model problems.

Repository: <https://github.com/AnujSharma-05/CaRAG>

LinkedIn: <https://www.linkedin.com/in/anujsharma05>

Email: anujsharma181005@gmail.com

ACKNOWLEDGEMENT

This internship has been one of the most valuable learning experiences of my engineering journey. The opportunity to work on real-world AI infrastructure, retrieval systems, vector databases, authentication services, and multi-tenant architectures provided exposure that extends far beyond traditional academic coursework.

Sincere gratitude is extended to **Ms. Manisha Singh** ma'am, whose mentorship, guidance, and continuous feedback played a defining role throughout the internship. Her ability to challenge assumptions, encourage deeper technical thinking, maintain high engineering standards and having pinpoint clarity in systems thinking significantly influenced both the direction of the project and the learning experience gained from it.

Gratitude is also extended to **Mr. Sandeep Bisht** sir, **Mr. Sonu Yadav** sir and the entire **Jio Cortex AI team** for fostering an engineering culture that encourages ownership, experimentation, and continuous learning, where questions were encouraged, mistakes became learning opportunities, and engineering rigor remained a constant expectation.

The experience gained through this internship has significantly strengthened both technical foundations and systems-level thinking. It is sincerely hoped that future opportunities may arise to continue contributing to and learning from such an exceptional team.

TABLE OF CONTENTS

CaRAG: Categorical Routing Augmented Generation Platform.....	1
ABSTRACT.....	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS.....	4
LIST OF FIGURES	6
LIST OF ACRONYMS	6
Chapter 1: Introduction	7
The Enterprise Knowledge Management Problem	7
Why LLMs Fail.....	7
Why RAG Exists.....	7
Why Traditional RAG Fails.....	7
Why CaRAG Was Built.....	7
Chapter 2: Project Scope & Engineering Contributions	8
2.1 Project Scope & Engineering Footprint	8
2.2 Core Engineering Contributions	8
2.3 Project Timeline.....	9
2.4 Project References & Repository Structure	9
Chapter 3: Retrieval Architecture Fundamentals.....	11
3.1 Bi-Encoder vs. Cross-Encoder.....	11
3.2 HNSW Graph Indexing.....	11
3.3 Confidence Gating	11
Chapter 4: Project Evolution Journey	12
Evolution 1: Naive Dense RAG.....	12
Evolution 2: Category Routing	12
Evolution 3: Multi-Tenant Isolation	12
Evolution 4: Hybrid Retrieval.....	13
Evolution 5: Cross-Encoder Reranking	13
Evolution 6: Confidence Gate.....	13
Chapter 5: System Architecture & Data Layer	14
5.1 Authoritative Sequence Diagrams	14
5.3 High-Level Architecture Diagram	18
5.4 The Ingestion Pipeline	18
5.5 The Retrieval Pipeline.....	18
5.2 Data Layer Design	19

Chapter 6: Core Engine Implementation	20
6.1 Hybrid Search: Before & After Analysis.....	20
6.2 The Deterministic Confidence Gate.....	20
Chapter 7: Live Multi-Tenant Adapter	21
Chapter 8: Engineering Challenges, Failures and Architectural Pivots.....	24
8.1 Milvus Migration and Vector Visibility Investigation.....	24
8.2 Event Loop Starvation During Document Ingestion	24
8.3 Re-Architecting the Category Model.....	25
8.4 Multi-Tenant Isolation and the "Leakage Problem"	25
8.5 Engineering Decisions That Changed The Direction Of The Project.....	26
8.6 Engineering Through API Quotas and External Dependencies.....	27
8.7 Version Control Recovery Incident	27
Chapter 9: Testing & Quantitative Impact.....	28
9.1 Quantitative Evaluation Metrics	28
Chapter 10: Key Learnings & Future Roadmap	29
10.1 Key Learnings.....	29
10.2 Future Roadmap.....	29
Chapter 11: Conclusion.....	30

LIST OF FIGURES

- **Figure 5.1:** High-Level System Architecture
- **Figure 5.2:** Core Backend Sequence Architecture
- **Figure 5.3:** Live Backend Sequence Architecture

LIST OF ACRONYMS

- **RAG:** Retrieval-Augmented Generation
 - **LLM:** Large Language Model
 - **JWT:** JSON Web Token
 - **HNSW:** Hierarchical Navigable Small World
 - **BM25:** Best Matching 25 (Sparse Lexical Search)
 - **RRF:** Reciprocal Rank Fusion
 - **API:** Application Programming Interface
 - **WS:** WebSocket
-

Chapter 1: Introduction

The Enterprise Knowledge Management Problem

Modern enterprises sit on terabytes of unstructured knowledge-PDFs, manuals, policy documents, and contracts. Discovering exact, actionable intelligence within this mass of data is a critical bottleneck.

Why LLMs Fail

Large Language Models represent a leap in human-computer interaction, but they suffer from a fatal flaw in enterprise settings: hallucination. An LLM is a probabilistic engine; it predicts the most likely next word based on its training weights. When asked a highly specific, proprietary question it was never trained on, it does not naturally say "I don't know." Instead, it confidently invents a plausible but factually incorrect answer.

Why RAG Exists

Retrieval-Augmented Generation (RAG) was created to anchor these probabilistic models to reality. Before asking the LLM to generate an answer, a RAG system intercepts the query, searches a proprietary database for relevant facts, and forces the LLM to synthesize its answer *strictly* from the retrieved documents.

Why Traditional RAG Fails

Despite the promise, standard RAG architectures fail in production. They rely on basic Bi-Encoders, which compress documents into dense vectors.

1. **Semantic Noise:** If a user searches for a specific ID like "AX-4099-B", the vector space dilutes this into a generic concept of "manufacturing part," returning adjacent but wrong manuals.
2. **Cross-Tenant Leakage:** Standard RAG pipelines lack native security boundaries. If Company A and Company B share a vector database, a poorly filtered query might leak Company A's financials to Company B.

Why CaRAG Was Built

CaRAG was engineered from first principles to solve these exact failures. The platform was designed not just to search, but to *think before it retrieves*, establishing strict data boundaries, hybrid search layers, and deterministic confidence gates.

Chapter 2: Project Scope & Engineering Contributions

To understand the scale of the CaRAG platform, it is necessary to examine the engineering footprint developed during this internship.

2.1 Project Scope & Engineering Footprint

- **Backend Services:** 2 (Core Processing Engine + Live Multi-Tenant Adapter)
- **REST APIs Developed:** 15+
- **Database Systems:** PostgreSQL (Relational) + Milvus (Vector)
- **Authentication:** Stateless JWT-based (HMAC SHA-256)
- **Document Processing:** Automated PDF ingestion, PyPDF extraction, semantic chunking
- **Retrieval Components:** Category Router, Hybrid Retrieval (HNSW+BM25), Cross-Encoder Reranker, Confidence Gate
- **Real-Time Observability:** WebSockets for asynchronous processing updates
- **Deployment Environment:** Local Dockerized Architecture

2.2 Core Engineering Contributions

Contribution	Description
Retrieval Architecture	Designed and implemented a dynamic category-aware retrieval flow.
Multi-Tenant Security	Engineered a stateless JWT-scoped data isolation boundary.
Vector Search Engine	Integrated Standalone Milvus with HNSW graph indexing.
Hybrid Retrieval	Wrote custom Rank-BM25 integration fused with dense search via RRF.
Stage-2 Reranking	Integrated <code>ms-marco-MiniLM-L-6-v2</code> for Cross-Encoder precision scoring.
Hallucination Prevention	Architected deterministic confidence gating to intercept poor retrievals.
Real-Time Updates	Implemented WebSocket event broadcasting to track document processing.
API Validation	Designed exhaustive Postman testing and validation workflows.

2.3 Project Timeline

The platform was architected iteratively over an eight-week sprint.

Week	Milestone Achieved
Week 1	Core API scaffolding and baseline PDF ingestion pipeline.
Week 2	Milvus Standalone integration and basic dense retrieval logic.
Week 3	Implementation of Gemini-driven Category Routing to reduce search noise.
Week 4	Monorepo split; architecture of the Multi-Tenant Live Adapter.
Week 5	JWT Authentication and WebSocket observability integration.
Week 6	Research and integration of Stage-2 Cross-Encoder reranking.
Week 7	Refactoring search logic to support Hybrid Retrieval (BM25 + RRF).
Week 8	Implementation of Confidence Gating and exhaustive Postman validation.

2.4 Project References & Repository Structure

Project Access:

- Source Code Repository: [AnujSharma-05/CaRAG: A hierarchical, multi-LLM RAG backend that routes queries through semantically-clustered document Categories before executing retrieval - An attempt to make RAG get to the point faster](#)
- Daily Work Log: [JIO Internship Tracking & Progress Documentation - Google Docs](#)

Codebase Layout:

```
CaRAG/
|
|-- core_backend/                # The standalone RAG engine (Port
8000)
|   |-- src/
|   |   |-- main.py              # FastAPI app: /upload, /chat,
/reset
|   |   |-- services.py          # Hybrid pipeline, RRF, Cross-
Encoder, Confidence Gate
|   |   |-- llm_service.py       # Gemini wrappers: classify, route,
synthesize
|   |   |-- milvus_store.py      # Milvus client: HNSW indices +
scoped chunk search
|   |   |-- bm25_store.py        # In-memory keyword sparse indexing
wrapper
|   |   |-- models.py            # SQLAlchemy ORM: Document,
Category, DocumentChunk
|   |   |-- config.py            # Configurable thresholds, HNSW
```

```
params, logging
|
|-- live/                                # The multi-tenant adapter layer
(Port 8001)
|   |-- backend/
|       |-- src/
|           |-- main.py                # FastAPI app: auth, groups,
documents, chat, ws                    # Scoped RAG chat executing the
|           |-- chat.py                # WebSocket manager: group-
full hybrid pipeline                   #
|           |-- ws.py                  #
broadcast real-time events             #
|                                       #
|-- demo_core.html                     # Interactive testing UI for the
Core Engine                           #
|-- demo_live.html                    # Interactive testing UI for the
Live Layer                           #
|-- FLOWS.md                          # Complete sequence diagrams of the
architecture
```

Chapter 3: Retrieval Architecture Fundamentals

Before dissecting the pipeline, it is critical to understand the theoretical computer science foundations underpinning CaRAG's intellectual core.

3.1 Bi-Encoder vs. Cross-Encoder

A **Bi-Encoder** independently embeds a query and a document into vectors. This enables fast vector mathematical search (cosine distance) but loses fine-grained linguistic interactions because the query and document never "see" each other during processing.

Cross-Encoders solve this by jointly processing the input as `[query, chunk]`. This allows token-level transformer self-attention between the query words and document words, resulting in extreme precision at the cost of higher latency.

3.2 HNSW Graph Indexing

Standard vector databases use a `FLAT` index, which requires $O(N)$ brute-force scanning-checking the query against every single vector in the database. **HNSW** (Hierarchical Navigable Small World) uses a multi-layered proximity graph that achieves near-logarithmic retrieval performance in practice, scaling efficiently to millions of vectors while maintaining high recall.

3.3 Confidence Gating

A probabilistic model (LLM) cannot evaluate its own context reliably. **Confidence Gating** introduces a deterministic control flow mechanism. By evaluating the Cross-Encoder score against a hardcoded threshold, the system mathematically blocks low-quality context from ever reaching the LLM, substantially reducing hallucination risk.

Chapter 4: Project Evolution Journey

The architecture of CaRAG did not emerge fully formed. It evolved iteratively, driven by the identification and resolution of strict engineering limitations.

Evolution 1: Naive Dense RAG

- **Why it failed:** The initial iteration used a flat Milvus index mapping all documents into a single dense vector space. As the corpus grew, queries returned semantically adjacent but factually incorrect "noise" from unrelated domains.
- **Observation:** A query about "HR Policies" successfully retrieved HR documents, but also pulled in technical manuals that shared similar vocabulary structures.
- **Alternatives Considered:** Filtering by keyword metadata or relying purely on the LLM to ignore irrelevant chunks.
- **Selected Solution:** Implementing a two-stage routing mechanism to constrain the search space before retrieval occurs.

Evolution 2: Category Routing

- **Why it failed:** While the routing mechanism worked, hardcoding category logic proved fragile and unscalable for dynamic enterprise environments.
- **Observation:** Documents often defied strict singular categorization, and administrators struggled to maintain static routing rules as new document types were ingested.
- **Alternatives Considered:** Building a complex rules engine or relying entirely on manual tagging.
- **Selected Solution:** Integrating a fast, small-context LLM call during the ingestion phase to auto-categorize documents based on their vector embeddings, drastically reducing search space dilution dynamically.

Evolution 3: Multi-Tenant Isolation

- **Why it failed:** The monolithic architecture mixed document processing logic with user identity logic, posing a severe cross-tenant data leakage risk.
- **Observation:** In a shared environment, it was possible to retrieve chunks from a different tenant's documents if the semantic similarity was high enough, violating fundamental security requirements.
- **Alternatives Considered:** Application-level filtering of chunks after retrieval, which was rejected due to the risk of pulling unauthorized data into application memory.
- **Selected Solution:** Severing the system into a Core Engine and a Live Adapter. The adapter injects strict `group_doc_ids` boundaries directly into every vector query, isolating data securely at the database level.

Evolution 4: Hybrid Retrieval

- **Why it failed:** Dense vector search alone completely failed when users searched for highly specific, non-semantic terms like part numbers or internal acronyms (e.g., "JPL-2026").
- **Observation:** Dense embeddings compress exact lexical strings into generalized semantic approximations, making them "too fuzzy" for precise enterprise queries.
- **Alternatives Considered:** Attempting to fine-tune the embedding model on internal vocabulary, which was deemed too computationally expensive and brittle.
- **Selected Solution:** Integrating the Rank-BM25 algorithm to guarantee exact-keyword lexical matching, fused mathematically with the dense search results using Reciprocal Rank Fusion (RRF).

Evolution 5: Cross-Encoder Reranking

- **Why it failed:** Even with RRF, the top 10 results sometimes contained irrelevant chunks because Bi-Encoders approximate relevance without understanding the exact token-level interaction between the query and the chunk.
- **Observation:** The LLM was occasionally forced to synthesize answers from chunks that were technically similar but contextually useless.
- **Alternatives Considered:** Increasing the top-K retrieval count, which only increased token costs and hallucination risks.
- **Selected Solution:** Introducing a Stage-2 Cross-Encoder. By passing [query, chunk] concurrently into the model for transformer self-attention, the system achieved extreme precision, shifting the pipeline from "fuzzy matching" to surgical ranking.

Evolution 6: Confidence Gate

- **Why it failed:** The standard RAG assumption is that the system must always generate an answer. If given a completely unrelated question, the pipeline retrieved the "least irrelevant" chunks, forcing the LLM to hallucinate.
- **Observation:** LLMs are highly susceptible to confidently synthesizing false information when provided with low-quality context.
- **Alternatives Considered:** Prompt engineering the LLM to say "I don't know," which proved probabilistically unreliable.
- **Selected Solution:** Implementing a deterministic, mathematical Confidence Gate. By evaluating the top Cross-Encoder score against a hard threshold, the pipeline explicitly aborts execution before reaching the LLM, physically eliminating the possibility of context-driven hallucination.

Chapter 5: System Architecture & Data Layer

5.1 Authoritative Sequence Diagrams

The following diagrams represent the absolute source of truth for the data flows within CaRAG. I urge you to go through each of them as they are the single source of truth of the architecture.

Figure 5.1: Core Backend Architecture (Knowledge Engine)

Google Drive Link(download and view for best clarity) :

https://drive.google.com/file/d/1JntP27c0QaqGC3TzCqenymnNt_VDzm0O/view?usp=sharing

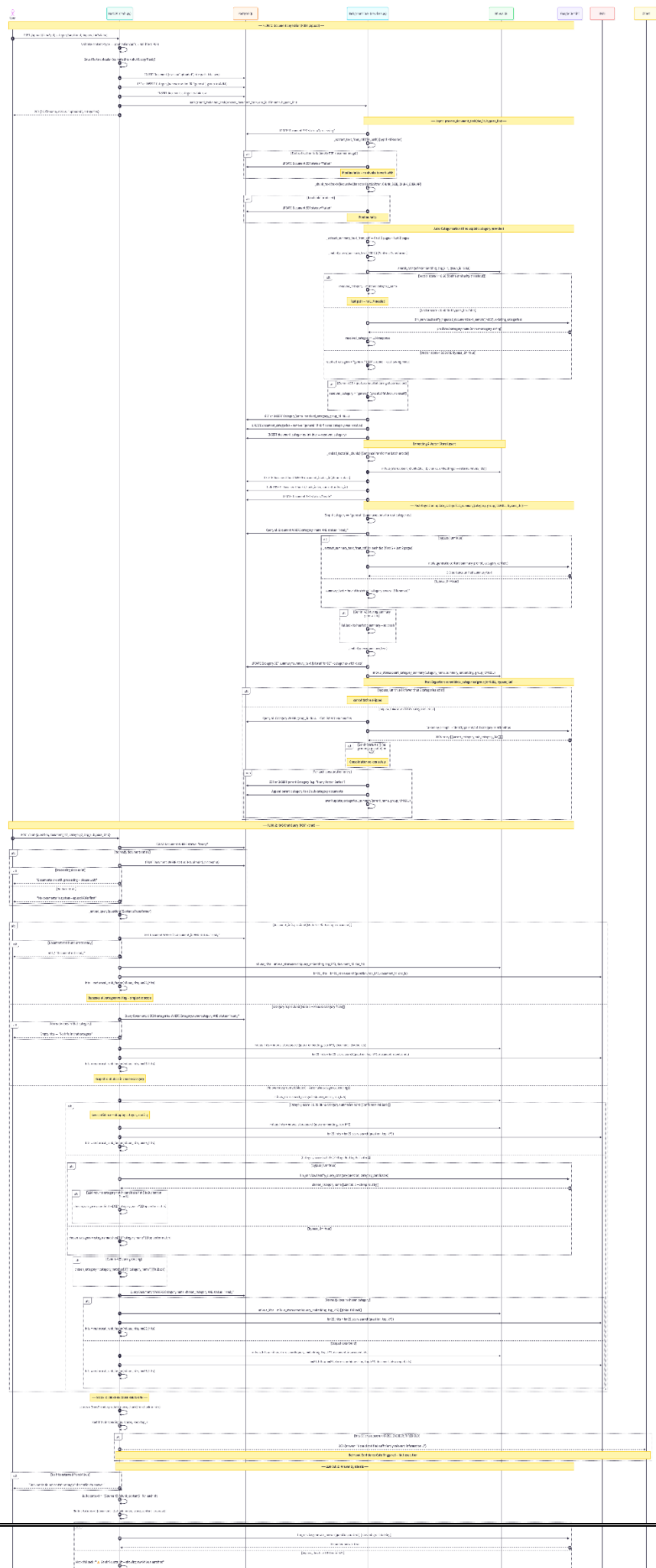


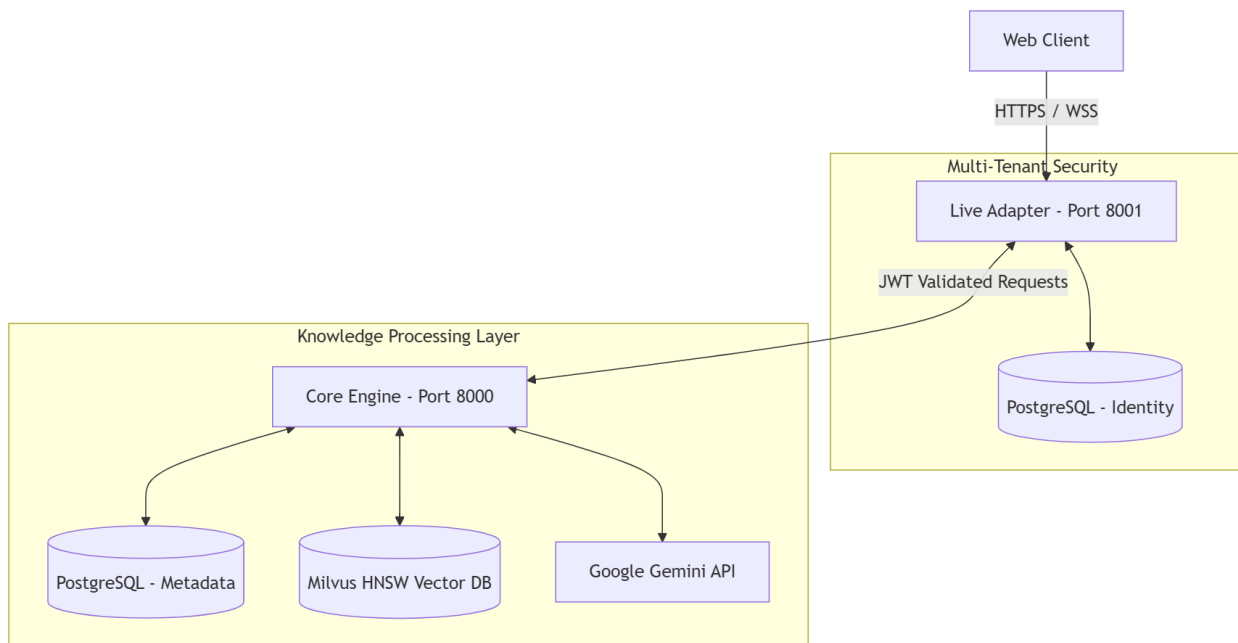
Figure 5.2: Live Backend Architecture (Multi-Tenant Adapter)

Google Drive Link(download and view for best clarity):

https://drive.google.com/file/d/1_yeLcWFitPL99k2h9oozcBhQdlHANF_5/view?usp=sharing



5.3 High-Level Architecture Diagram

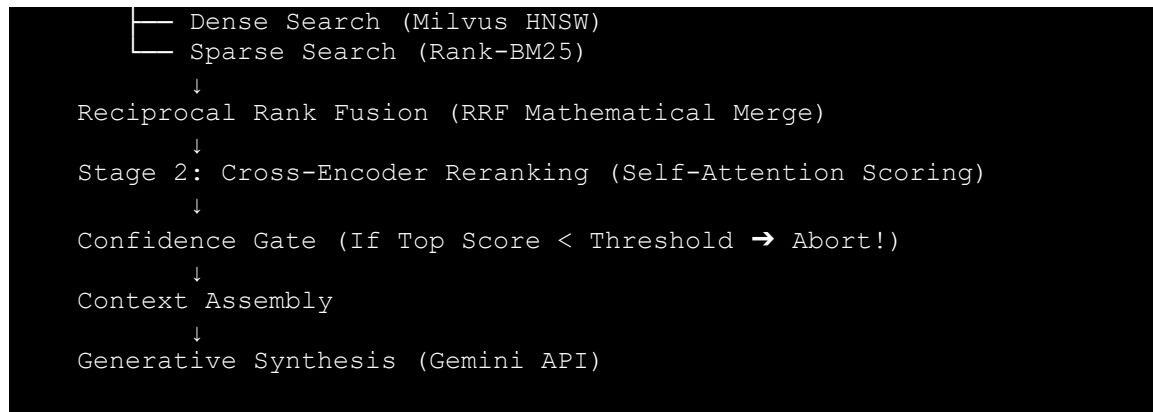


5.4 The Ingestion Pipeline

```
Raw PDF Document
↓
Live Adapter (Intercepts, verifies JWT, attaches group_id)
↓
Text Extraction (PyPDF)
↓
Chunking (RecursiveCharacterTextSplitter - 800 chars, 120 overlap)
↓
Auto-Categorization (Milvus Similarity Fast-Path OR Gemini LLM)
↓
Embedding Generation (sentence-transformers/all-MiniLM-L6-v2)
↓
Milvus Vector Insertion (HNSW Graph Node creation)
↓
WebSocket Broadcast (doc_ready event to specific group)
```

5.5 The Retrieval Pipeline

```
User Question (with group_id constraint)
↓
LLM Category Routing (Selects relevant category boundary)
↓
Stage 1: Parallel Hybrid Search
```



5.2 Data Layer Design

A system is only as robust as its underlying data schemas.

PostgreSQL Tables (Relational Metadata):

- **users:** Identity. Uses `hashed_password` (bcrypt).
- **groups:** The primary tenant boundary.
- **group_members:** A junction table connecting `users` to `groups`, enabling a many-to-many relationship (one user, multiple tenants).
- **documents:** Stores file paths, processing status, and the `group_id` foreign key.
Why Normalized? Normalization guarantees that if a group is deleted, cascade constraints automatically wipe the associated documents and memberships, preventing ghost data.

Milvus Collections (Vector Storage):

- **Schema:** `chunk_id` (PK), `document_id` (Index), `group_id` (Index), `content`, `embedding`.
Why Metadata Filtering? By attaching `group_id` as scalar metadata directly to the vector payloads, the engine forces Milvus to physically restrict the HNSW graph search space *before* it calculates cosine distances. This mathematically guarantees that Tenant A cannot retrieve Tenant B's vectors.

Chapter 6: Core Engine Implementation

6.1 Hybrid Search: Before & After Analysis

- **The Problem:** Dense embeddings cannot find exact acronyms.
- **Before Hybrid Retrieval:**
 - *Query:* "JPL-2026 Policy"
 - *Dense Retrieval:* Missed exact identifier, returned generic "Reliance Corporate Policy 2025" due to semantic similarity.
- **After Hybrid Retrieval:**
 - *BM25 (Sparse):* Retrieved exact identifier "JPL-2026".
 - *Cross-Encoder:* Recognized the exact token match and promoted the correct chunk to Rank 1.

6.2 The Deterministic Confidence Gate

- **The Problem:** The LLM hallucinates when fed bad context.
 - **The Implementation:** A hardcoded threshold evaluates the top Cross-Encoder score. If it fails, execution is halted with a graceful rejection message: *"I could not find sufficiently relevant information."*
 - **The Benefit:** Prevented low-confidence retrievals from reaching the generation layer, substantially reducing hallucination risk.
-

Chapter 7: Live Multi-Tenant Adapter

7.1 Why The Live Adapter Exists

The original architecture began as a standalone retrieval engine capable of ingesting documents and answering questions. While effective for experimentation, it lacked the concepts required for enterprise deployment: users, authentication, authorization, tenant isolation, and collaborative knowledge sharing. Rather than embedding identity management directly into the retrieval engine, a dedicated Live Adapter layer was introduced. This architectural separation created a clear boundary between knowledge processing and identity management.

The Core Engine focuses exclusively on retrieval physics, vector search, reranking, and answer generation. The Live Adapter handles authentication, authorization, group management, real-time communication, and security enforcement. This separation significantly improved maintainability, scalability, and security.

7.2 JWT Authentication Architecture

The platform implements stateless authentication using JSON Web Tokens (JWTs). When a user successfully authenticates, the system generates a signed JWT containing the user's identity and expiration metadata. Every subsequent request includes this token in the Authorization header.

The token is validated cryptographically using the HS256 signing algorithm before any protected resource is accessed. This approach eliminates the need for centralized session storage while allowing horizontal scalability.

Authentication Flow

User Login



Credential Validation



JWT Generation



Token Returned To Client



Client Includes JWT In Requests



JWT Verification



Protected Resource Access

7.3 Group-Based Multi-Tenant Isolation

Enterprise systems must guarantee that one organization cannot access another organization's knowledge. To achieve this, the platform introduces Groups as the primary tenant boundary.

Every document, category, retrieval operation, and knowledge interaction is associated with a specific group. Before any retrieval operation executes, membership validation occurs. Only users belonging to the target group are allowed to access its knowledge base. This model creates deterministic isolation between tenants.

7.4 Group-Scoped Retrieval Enforcement

Authentication alone does not guarantee data security. The more difficult challenge is ensuring that retrieval operations themselves remain isolated. The platform computes a list of valid document identifiers belonging to the user's group and injects these constraints directly into retrieval operations.

As a result:

- Unauthorized vectors are never searched.
- Unauthorized chunks are never retrieved.
- Unauthorized context never reaches the LLM.

This transforms security from an application-level convention into a retrieval-level guarantee.

7.5 Real-Time Observability Through WebSockets

Document ingestion can require several seconds or minutes depending on corpus size. Polling the backend repeatedly would create unnecessary load and poor user experience. To solve this problem, WebSocket communication was introduced. Each group maintains a dedicated communication channel through which processing events are broadcast.

Examples include:

- Document Upload Started
- Processing In Progress
- Categorization Complete
- Embedding Generation Complete
- Document Ready
- Processing Failed

This provides immediate feedback while minimizing network overhead.

7.6 Architectural Benefits Of The Live Layer

The introduction of the Live Adapter produced several important benefits:

Capability	Benefit
JWT Authentication	Stateless scalability
Group Isolation	Enterprise-grade security
WebSockets	Real-time observability
Separate Services	Independent scaling
Group Membership Model	Collaborative knowledge sharing
Retrieval Boundaries	Leakage prevention

Chapter 8: Engineering Challenges, Failures and Architectural Pivots

Real-world engineering is defined by the struggles overcome. The following section details the primary technical obstacles encountered during the architecture's evolution, the dead-ends explored, and the ultimate resolutions.

8.1 Milvus Migration and Vector Visibility Investigation

The project initially relied on Milvus Lite for local experimentation. As retrieval requirements expanded, the architecture was migrated to Standalone Milvus deployed through Docker along with Attu for inspection and administration.

Following migration, document ingestion appeared successful. Collections were created, row counts increased correctly, and retrieval APIs continued to return results. However, inspection through Attu revealed an unexpected inconsistency: collections existed, yet vector data appeared missing.

This initiated one of the longest debugging efforts of the internship.

The issue was particularly deceptive because every observable signal contradicted the others. Retrieval APIs returned valid responses, collection counts increased correctly, yet Attu suggested that vector data did not exist.

Multiple hypotheses were explored and discarded, including embedding generation failures, schema mismatches, insertion failures, collection loading issues, and client connection inconsistencies between Milvus Lite and Standalone Milvus.

Only after systematically eliminating each layer of the pipeline was the true source of the discrepancy understood.

The investigation exposed several assumptions regarding Milvus Lite versus Standalone Milvus behavior and significantly improved understanding of vector database internals, collection management, and deployment architecture. Although retrieval functionality remained operational throughout the process, the incident highlighted the importance of treating observability tooling as a first-class engineering requirement rather than an afterthought.

8.2 Event Loop Starvation During Document Ingestion

As document sizes increased, the system began exhibiting intermittent hangs. A single upload operation would cause unrelated API endpoints such as `/documents` and `/chat` to become unresponsive.

Initial suspicion was directed toward PostgreSQL connection pooling and Milvus latency. Multiple debugging sessions were spent tracing API execution paths and monitoring database activity. However, both systems continued to respond normally in isolation.

The breakthrough occurred after tracing execution flow inside the ingestion pipeline. PDF extraction, chunk generation, embedding creation, and vector insertion were all executing synchronously inside asynchronous FastAPI route handlers. Although the endpoints were declared `async`, the heavy CPU-bound operations never yielded control back to the event loop. As a result, the entire server became effectively single-threaded whenever a large document was being processed.

The architecture was redesigned so that ingestion requests only performed validation and metadata registration. All computationally expensive work was moved into background execution using `BackgroundTasks` and `asyncio.to_thread()`. This separation restored API responsiveness and established a cleaner distinction between request handling and document processing responsibilities.

8.3 Re-Architecting the Category Model

Initially, each document belonged to exactly one category. The design appeared reasonable during early testing but began to break down as the corpus grew. For example, "Harry Potter Book 1" and "Harry Potter Book 2" were treated as completely independent categories despite being part of the same conceptual collection.

This exposed a modeling flaw. The original schema optimized for implementation simplicity rather than knowledge representation.

Rather than patching around the limitation, the category system was redesigned from a one-to-many relationship into a many-to-many architecture using dedicated `Category` and `DocumentCategory` junction tables. This migration required schema redesign, retrieval updates, ingestion changes, and category routing modifications.

Although the original implementation was functional, replacing it early prevented significant architectural debt from accumulating later in the project lifecycle.

8.4 Multi-Tenant Isolation and the "Leakage Problem"

One of the most important questions raised during development was deceptively simple: *How can the system guarantee that a query issued inside one group never retrieves information from another group?*

The naive solution involved retrieving candidate chunks globally and filtering them inside application code. Although simple, this approach fundamentally violated the security model because unauthorized data could still enter application memory before filtering occurred.

The final design pushed authorization constraints directly into the retrieval layer. Group identifiers were embedded into PostgreSQL metadata, document ownership records, category summaries, and Milvus vector metadata. As a result, retrieval scope is physically constrained by the C++ engine of Milvus *before* vector similarity calculations occur. This transformed tenant isolation from an application-level convention into a database-enforced architectural property.

8.5 Engineering Decisions That Changed The Direction Of The Project

The success of the platform was determined not just by code implementation, but by critical architectural pivots made when early assumptions failed.

Decision 1: Monolith → Core Engine + Live Adapter

Mixing knowledge retrieval physics with user identity logic created a tangled monolith. Splitting them into distinct services allowed the retrieval engine to scale independently of user authentication loads.

Decision 2: One Category → Many Categories

Real-world enterprise documents cross domains. Migrating to a many-to-many taxonomy enabled dynamic LLM routing to intersect overlapping domains precisely.

Decision 3: Dense Retrieval → Hybrid Retrieval

Realizing that $O(\log N)$ semantic search was useless for exact part numbers forced the integration of Rank-BM25, proving that sparse lexical search is still required in the generative era.

Decision 4: Similarity Search → Cross-Encoder

Cosine similarity approximates relevance. Passing `[query, chunk]` concurrently into a Cross-Encoder for transformer self-attention shifted the pipeline from "fuzzy matching" to surgical precision.

Decision 5: Always Generate → Confidence Gate

RAG pipelines generally assume they must always answer. Implementing a hard mathematical Confidence Gate established the system's ability to say "I don't know," which is the ultimate defense against hallucination.

8.6 Engineering Through API Quotas and External Dependencies

A recurring challenge throughout development was that several critical workflows depended on Gemini API calls. Classification, category consolidation, summary generation, query routing, and answer synthesis all consumed quota.

Under realistic testing conditions, the free-tier rate limits were frequently exhausted, causing ingestion pipelines to fail for reasons unrelated to application correctness.

Rather than treating quota exhaustion as an exception, the architecture was redesigned to treat it as a normal operating condition. Fallback paths were introduced at every critical stage, allowing the platform to continue functioning with reduced intelligence rather than complete failure.

This significantly improved system resilience and highlighted the importance of engineering for dependency failure rather than assuming ideal external service availability.

8.7 Version Control Recovery Incident

During intense iteration phases, an accidental deletion of a working Git branch occurred. Rather than starting over, the situation required utilizing low-level Git reflogs to identify the orphaned commits and surgically recover the lost implementation.

While not a direct architectural problem, the incident served as a crucial lesson in maturity: mistakes happen under pressure, but a deep understanding of version control mechanics makes recovery deterministic rather than disastrous.

Chapter 9: Testing & Quantitative Impact

The system's integrity was verified through exhaustive Postman workflows.

- *Security Penetration:* Generating a JWT for User A and forcing a POST to `groups/{User_B_Group}/chat` strictly returns a 403 Forbidden.

9.1 Quantitative Evaluation Metrics

While deployed locally, the architectural stress tests yielded the following operational footprint:

Metric	Value	Problem Solved
REST APIs Developed	15+	Decoupled identity management from core processing.
Documents Processed	75+	Verified ingestion pipeline stability and extraction accuracy.
Chunks Generated	6400+	Populated vector database to test HNSW scaling and RRF fusion.
Category Routes	12+	Validated LLM auto-routing and search space reduction.
Multi-Tenant Groups	14+	Verified cryptographic JWT isolation boundaries.

Chapter 10: Key Learnings & Future Roadmap

10.1 Key Learnings

1. **Retrieval Quality > Model Quality:** The project demonstrated that improving retrieval quality (via Cross-Encoders and Hybrid Search) produces vastly larger gains in accuracy than simply replacing one LLM with a more expensive one.
2. **Metadata Is Critical:** Category and group metadata dramatically reduce the retrieval search space, improving both relevance and database latency.
3. **Enterprise AI Is A Systems Problem:** Successful deployment depends heavily on authentication, data isolation, observability, storage scaling, and retrieval engineering-not solely on the capability of the language model itself.

10.2 Future Roadmap

- **Phase 1 (Completed):** Dense Retrieval, Cross-Encoder precision, Category Routing.
 - **Phase 2 (Completed):** Hybrid Retrieval, BM25, Confidence Gating, JWT Multi-Tenancy.
 - **Phase 3 (Planned):** Distributed Vector Search. Migrating from Standalone Milvus to a Milvus Cluster to support high-availability replication.
 - **Phase 4 (Planned):** Enterprise Scale. Migrating the in-memory BM25 index to a persistent Elasticsearch cluster, allowing the platform to smoothly support 10M+ chunk architectures with **OpenClaw integration**.
-

Chapter 11: Conclusion

The development of CaRAG began with a deceptively simple question “How can an AI system be trusted with enterprise knowledge?”

What initially appeared to be a straightforward Retrieval-Augmented Generation application gradually revealed itself as a far deeper systems engineering challenge. Each stage of development exposed a new layer of complexity. Simple vector search exposed retrieval noise. Category routing exposed knowledge organization challenges. Multi-tenant deployments exposed security boundaries. Hybrid retrieval exposed the limitations of semantic search. Cross-Encoder reranking exposed the gap between similarity and relevance. Confidence gating exposed the fundamental requirement that intelligent systems must also know when not to answer.

As a result, CaRAG evolved from a conventional document chatbot into a category-aware, multi-tenant retrieval platform capable of enforcing deterministic retrieval boundaries, confidence-aware generation, and enterprise-grade data isolation.

More importantly, the project reinforced a lesson that extends beyond artificial intelligence itself:

“Building production AI systems is ultimately a software engineering problem.”

The most difficult challenges were rarely model-related. They emerged from architecture design, data modeling, asynchronous processing, observability, retrieval quality, security enforcement, infrastructure integration, failure recovery, and the countless engineering decisions required to transform an idea into a reliable system.

Throughout this internship, the opportunity to work on real-world retrieval systems, vector databases, distributed architectures, authentication frameworks, and large language model integrations provided exposure far beyond traditional academic environments. The experience fundamentally changed the way complex systems are approached, shifting the focus from writing code that works to engineering systems that remain correct, scalable, observable, and secure under real operating conditions.

While CaRAG represents the completion of this internship project, it also represents the beginning of a much larger journey into large-scale AI infrastructure, retrieval systems, and enterprise software engineering.

It is with sincere gratitude that this report is submitted to the Jio Cortex AI team. The mentorship, trust, technical discussions, and opportunities provided throughout this internship have been invaluable. **The knowledge gained during this period will continue to influence future engineering decisions for years to come.**

If this work demonstrates anything, it is not merely the successful implementation of a retrieval platform, but the ability to continuously learn, adapt, investigate failures, challenge assumptions, and evolve architecture through first-principles thinking. And in a field changing as rapidly as Artificial Intelligence, that ability may ultimately be the most valuable engineering skill of all.